# Intro

- 徐卓一 10235102432
- 朱昭荣 10234304416
- 分工：徐卓一完成TTL部分以及测试汇报，朱昭荣完成metadata维护部分
  测试：

```
1. 配置与构建
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug \
  -DLEVELDB_BUILD_TESTS=ON \
  -DLEVELDB_BUILD_BENCHMARKS=OFF

cmake --build build -j2
2. 课程专项测试
(cd build && ./ttl_test)
(cd build && ./metadata_test)
3. 补充测试与完整回归
# 运行全部内部测试，包含新增的 TTL、统计和 MANIFEST 测试
(cd build && ./leveldb_tests)

# 完整 CTest
mkdir -p build/test-data
TEST_TMPDIR="$PWD/build/test-data" \
  ctest --test-dir build --output-on-failure
4. 联合快速评测
python3 benchmark/runner.py --kind combined --source "$PWD" \
  --output "$PWD/build/evaluation-combined-quick-$(date +%Y%m%d-%H%M%S)" \
  --profile quick --jobs 2
5. 正式性能评测
python3 benchmark/runner.py --kind combined --source "$PWD" \
  --output "$PWD/build/evaluation-combined-standard-$(date +%Y%m%d-%H%M%S)"
\
  --profile standard --matrix --jobs 2
```

# 设计思路和实现过程

## 阶段一 TTL 写入

整体流程，都从四参数用户调用开始，在这整个流程中添加属于TTL的处理逻辑

```
db->Put(options, key, value, ttl)
└─ DBImpl::Put(options, key, value, ttl)       db/db_impl.cc:1245(TODO)
   │
   ├─ ComputeExpirationTime(...) (TODO)
   │    └─ 将相对 TTL 转为绝对过期时间 expiration_micros
```

```
    |
    ├─ WriteBatchInternal::PutWithTTL(...)     db/write_batch.cc:126(TODO)
    |    ├─ EncodeValueWithTTL(...)
    |    └─ 把 TTL 操作编码进 batch.rep_
    |
    └─ DBImpl::Write(options, &batch)          db/db_impl.cc:1260
         |
         |
         ├─ log_->AddRecord(...)              db/db_impl.cc:1292
         |    └─ 将整个 batch 写入 WAL
         |
         ├─ logfile_->Sync()                 db/db_impl.cc:1295
         |    └─ 仅 options.sync 为 true 时执行
         |
         ├─ WriteBatchInternal::InsertInto(...)  db/db_impl.cc:1301
         |    ├─ 创建 MemTableInserter
         |    └─ batch.Iterate(&inserter)      db/write_batch.cc:169
         |         ├─ 识别 kTypeValueWithTTL (TODO)
         |         ├─ DecodeValueWithTTL(...)
         |         └─ inserter.PutWithTTL(...) db/write_batch.cc:155(TODO)
         |              ├─ EncodeValueWithTTL(...)
         |              ├─ mem_->Add(sequence_, kTypeValueWithTTL, ...)
         |              |    ├─ 在 Arena 中分配并编码记录
         |              |    └─ table_.Insert(buf)
         |              |         └─ 插入 MemTable 的跳表
         |              └─ sequence_++
         |
         ├─ 更新 LastSequence，唤醒等待的写线程
         └─ 返回 Status
```

在这一阶段，我们需要修改多个PUT函数，但是这些PUT函数在不同类中，扮演的角色是完全不同的，包括最上层负责用户调用的四参数接口db->Put，以及WriteBatchInternal::PutWithTTL(...)负责写writebatch，以及处理真正的写入逻辑的回调：inserter.PutWithTTL(...)

## 阶段二 Get 与快照时间

Get的起点依旧是用户调用接口，但着重修改的是分别Memtable的get和Version的get

```
Status DBImpl::Get(const ReadOptions& options, const Slice& key,
                   std::string* value) {
  Status s;
  MutexLock l(&mutex_);
  SequenceNumber snapshot;
  uint64_t read_time_micros;
  // Capture sequence and time together under mutex_, before reading any
source.
  if (options.snapshot != nullptr) {
```

```
    const auto* view = static_cast<const SnapshotImpl*>(options.snapshot);
    snapshot = view->sequence_number();
    read_time_micros = view->read_time_micros();
  } else {
    snapshot = versions_->LastSequence();
    read_time_micros = env_->NowMicros();
  }

  MemTable* mem = mem_;
  MemTable* imm = imm_;
  Version* current = versions_->current();
  mem->Ref();
  if (imm != nullptr) imm->Ref();
  current->Ref();

  bool have_stat_update = false;
  Version::GetStats stats;

  // Unlock while reading from files and memtables
  {
    mutex_.Unlock();
    // First look in the memtable, then in the immutable memtable (if any).
    LookupKey lkey(key, snapshot);
    #分别调用memtable的Get
    if (mem->Get(lkey, read_time_micros, value, &s)) {
      // Done
    } else if (imm != nullptr &&
               imm->Get(lkey, read_time_micros, value, &s)) {
      // Done
    #调用Version::Get
    } else {
      s = current->Get(options, lkey, read_time_micros, value, &stats);
      have_stat_update = true;
    }
    mutex_.Lock();
  }

  if (have_stat_update && current->UpdateStats(stats)) {
    MaybeScheduleCompaction();
  }
  mem->Unref();
  if (imm != nullptr) imm->Unref();
  current->Unref();
  return s;
}
```

我们的tag存储模式以及绝对过期时间存储位置

```
普通 Put
[key长度][用户key][sequence + kTypeValue][value长度][用户value]
TTL Put
[key长度][用户key][sequence + kTypeValueWithTTL][value长度][绝对过期时间：8字节]
[用户value]
TTL 情况下：
value长度 = 8 + 用户value长度;
Delete
[key长度][用户key][sequence + kTypeDeletion][value长度=0]
```

DMImpl（在这个get里就已经分别处理的snapshot的读取时间与否了，所以对应的snapshot的创建和类属性也要改变）的Get主要分为两种Get：memtable和version（SST）
对于memtable直接增加对case kTypeValueWithTTL的处理（memtable.cc）
对于version则需要对state.saver增加read_time_micros字段，然后在match函数的savevalue中处理

# 阶段三 迭代器

迭代器用于遍历，因为有TTL所以需要过滤过期记录；过滤时间选择 Snapshot 时间或迭代器创建时间

| 顺序 | 文件位置 | 实现的内容 |
| --- | --- | --- |
| 1 | `db/db_iter.h/.cc`，`NewDBIterator`/构造函数 | 增加固定读取时间参数和成员 |
| 2 | `db/db_impl.cc`，`NewIterator` | 选择 Snapshot 时间或迭代器创建时间 |
| 3 | `db/db_iter.cc`，`FindNextUserEntry` | 有效 TTL 解码，过期 TTL 按 deletion 跳过旧版本 |
| 4 | `db/db_iter.cc`，`FindPrevUserEntry` | 反向遍历实现同样语义 |

# 阶段四 写入时统计

一个 DBImpl 对象只有一个全局MetadataState metadata_

在dbimpl的write函数（会被Put，Delet等直接调用）中，在将writebatch写进之前，需要创建一个metadata_delta，然后通过CollectWriteBatchMetadata收集变化量，在CollectWriteBatchMetadata中：会指定PUT DELET PUTWITHTTL的回调处理函数为AddRecord，然后通过interate遍历writebatch，相当于对所有操作执行一遍addrecord，最终维护好了metadata_delta，再等WAL和memtable被写好之后，apply这个metadata_delt（更新全局metadata）

# 阶段五 全局统计恢复与 Compaction 回收

| 对象 | 类型 | 统计范围 |
| --- | --- | --- |
| DBImpl::metadata_ | MetadataState | 整个数据库 |
| MemTable::metadata_ | SourceMetadata | 当前这一个 MemTable |
| FileMetaData::metadata | SourceMetadata | 当前这一个 SST |

```
全局对象保存：
total_entries_
key_references_  // 用户键 → 内部记录数
局部对象
1.memtable metadata
struct SourceMetadata {
  uint64_t total_entries = 0;
  uint64_t distinct_keys = 0;
};

2.SST FileMetaData
  SourceMetadata metadata;
  bool metadata_known = false;  // Legacy MANIFEST records have no summary.
  std::shared_ptr<const FileKeyReferences> key_references;
  bool has_ttl = false;

所以这几个类的实例，都需要保存对应的属性
```

在MemTable::Add中（db/memtable.cc:77），增加一个判断，判断当前key是否已经存在于memtable中，如果不存在就distinct_keys++，否则只total_entries++

对于SST来说，关键函数是BuildTable（db/builder.cc），这个函数负责接收一个迭代器，然后将迭代器指向的内容写到SST中，这个过程一般对应memtable的flush，但因为flush的时候本身需要遍历一遍所有操作，所以可以重新统计一遍存入FileMetaData的新属性metadata里，但这个过程并不改变全局的metadata字段（db/db_impl.cc:531，WriteLevel0Table()在调用完BuildTable之后会检查memtable和SST的是否一致）

Memtable的metadata通过WAL重放执行PUT会被重新统计，SST的metadata则是被持久化到MANIFEST里（伴随FileMetaData被编码然后持久化）

当调用DBImpl的OPEN的时候，会先构造一个MetadataDelta rebuild，然后：

```
RebuildMetadata（尚未向用户开放 DB，也未启动后台合并）
  ├─ 遍历 current Version 的有效 SST
  │     └─ 从 FileMetaData.key_references 合并全局逐键引用数
  ├─ 扫描恢复后仍存活的 mem_/imm_ 内部记录，补齐其贡献
  └─ 一次Apply rebuild delta
```

对于compaction来说，首先对CompactionState增加属性

```
uint64_t compaction_time_micros = 0;
bool allow_ttl_gc = false;
MetadataDelta metadata_delta;
```

然后内部迭代器，用户键升序，同键序列号降序，然后判断是否可以drop，如果能drop则调用metadatadelta的RemoveRecord() 位于 db/metadata_stats.cc:40
等真的被回收删除并持久化之后再apply这个delta
重要：drop的判断方式

# 阶段六 补充测试

1. TTL 补充测试：28 个
   文件：db/ttl_internal_test.cc

- 时间与编码：绝对过期时间计算、溢出、二进制 value 编解码。
- WriteBatch：TTL 序列化、旧 Handler 兼容、损坏记录识别、混合操作类型和序列号。
- 写入与恢复：零 TTL、溢出不产生写入副作用、WAL/SST 恢复不延长有效期。
- 读取语义：到期边界、MemTable/immutable/SST 一致性、过期新版本不能暴露旧值。
- 快照与时钟：固定读取序列号和时间，验证时钟采样次数。
- 迭代器：Seek、正反向切换、固定视图、压实后视图、损坏处理，以及与独立模型对照的随机测试。

2. 统计元数据补充测试：32 个
   文件：db/metadata_stats_test.cc

- 计数规则：覆盖写、删除、TTL、混合 batch、二进制 key、自定义 Comparator。
- 原子性与故障：非法 batch、WAL 写入/同步失败不发布统计，并发查询不看到半个 batch。
- 局部与全局统计：MemTable/SST 摘要正确，刷盘不重复计数，跨数据源正确去重。
- 持久化与兼容：MANIFEST 重写、WAL/SST 恢复、旧格式升级、Repair 重建。
- 压实与 TTL 协同：精确扣减被回收记录，保护快照和迭代器，安装失败不扣减。
- 并发与异常恢复：写入、查询、压实并发一致性，非法持久化引用拒绝打开，同步写入后进程直接退出仍能恢复。

3. MANIFEST 编解码补充测试：4 个
   文件：db/version_edit_test.cc

- 新旧文件 metadata 混合编码与解码。
- 区分“已知为零”和“旧格式未知”。
- 拒绝截断或不一致的统计摘要。
- key 引用次数持久化往返，以及非法引用次数校验。

# 测试

我们运行

```
python3 benchmark/runner.py --kind combined --source "$PWD"   --output "$PWD/build/evaluation-combined-standard-$(date +%Y%m%d-%H%M%S)"   --profile standard --matrix --jobs 2
```

调用标准benchmark，这个benchmark本身就会重复循环三次，对应结果如下，168cases都通过

```
perf_ttl_mixed-n10000-t1-e100-r3
perf_ttl_mixed-n100000-t1-e0-r1
perf_ttl_mixed-n100000-t1-e0-r2
perf_ttl_mixed-n100000-t1-e0-r3
perf_ttl_mixed-n100000-t1-e50-r1
perf_ttl_mixed-n100000-t1-e50-r2
perf_ttl_mixed-n100000-t1-e50-r3
perf_ttl_mixed-n100000-t1-e100-r1
perf_ttl_mixed-n100000-t1-e100-r2
perf_ttl_mixed-n100000-t1-e100-r3
perf_ttl_write-n10000-t1-e50-r1
perf_ttl_write-n10000-t1-e50-r2
perf_ttl_write-n10000-t1-e50-r3
perf_ttl_write-n100000-t1-e50-r1
perf_ttl_write-n100000-t1-e50-r2
perf_ttl_write-n100000-t1-e50-r3
perf_write-n10000-t1-e50-r1
perf_write-n10000-t1-e50-r2
perf_write-n10000-t1-e50-r3
perf_write-n100000-t1-e50-r1
perf_write-n100000-t1-e50-r2
perf_write-n100000-t1-e50-r3
ttl_contract-n100000-t8-e50-r1
ttl_contract-n100000-t8-e50-r2
ttl_contract-n100000-t8-e50-r3
ttl_views-n100000-t8-e50-r1
ttl_views-n100000-t8-e50-r2
ttl_views-n100000-t8-e50-r3
{"status": "pass", "output": "/home/joeyxzy/db_class/leveldb_project1/build/evaluation-combined-standard-20261029-195401", "passed": 168, "cases": 168}
```

并让AI分析总结测试结果：

单独查询开销

下面均为三次运行的中位数，括号为吞吐量最小值～最大值。

| 数据规模 | 统计查询吞吐量（百万次/秒） | P99 延迟 |
|---|---|---|
| 10,000 | 3.460（3.291～3.522） | 0.046 µs |
| 100,000 | 3.232（3.227～3.251） | 0.046 µs |

数据量扩大 **10 倍**，查询吞吐仅下降约 *6.6%*，P99 基本不变。

这与实现方式一致：查询读取已维护的计数，而不是现场扫描所有记录。注意吞吐计时还包含驱动中的校验等开销，**不能简单将吞吐量倒数当作接口延迟。**

普通写入期间，统计查询的表现

| 数据规模 | 写线程数 | 统计查询吞吐（百万次/秒，中位数及范围） | P99 |
|---|---|---|---|
| 10,000 | 1 | 4.598（3.750～4.653） | 6.184 µs |
| 10,000 | 8 | 3.511（2.947～5.236） | 1.429 µs |
| 100,000 | 1 | 3.439（3.388～4.132） | 6.176 µs |
| 100,000 | 8 | 2.990（2.666～3.133） | 1.220 µs |

可以看出：

- 写入期间统计查询仍达到约 **299 万～460 万次/秒**。
- 相比静态查询，尾延迟明显增大，符合查询与写入共享互斥锁、受到调度影响的情况。
- 数据规模增大后查询吞吐下降，但不是随记录数线性下降。
- 不能根据这里 8 线程的 P99 更低，就断言"线程越多查询越快"；写入批处理、调度和锁占用模式也会变化。

统计查询对普通写入吞吐的影响

前三列吞吐量取三次中位数。最后一列是**每轮配对计算下降比例后，再取中位数**：

下降比例 = (1 − 有查询时写吞吐 / 无查询时写吞吐) × 100%

| 数据规模 | 写线程数 | 无查询写吞吐（次/秒） | 有查询写吞吐（次/秒） | 配对下降比例：中位数［范围］ |
|---|---|---|---|---|
| 10,000 | 1 | 206,890 | 188,110 | 9.08% [0.82%～14.44%] |
| 10,000 | 8 | 109,579 | 117,644 | -6.32% [-14.48%～0.82%] |
| 100,000 | 1 | 217,111 | 208,249 | 7.51% [3.18%～10.03%] |
| 100,000 | 8 | 67,475 | 69,050 | -2.33% [-14.49%～8.14%] |

**结论：**

- 单写线程时，持续查询统计使写吞吐下降约 *7.5%～9.1%*。
- 8 写线程时，结果有正有负、波动较大，**未观察到稳定的额外下降，也不能声称统计查询提高了写性能。**
- 8 线程总体吞吐反而低于单线程，说明该工作负载下增加写线程没有带来扩展收益；具体瓶颈需要进一步 profiling，不能只凭这些数据确定原因。

TTL 与统计协同场景

以 **50% 过期比例**为例：

| 数据规模 | 写线程数 | 无查询 TTL 写吞吐 | 有查询 TTL 写吞吐 | 配对下降比例：中位数［范围］ |
|---|---|---|---|---|
| 10,000 | 1 | 188,203 | 159,941 | 18.93% [15.02%～30.25%] |
| 10,000 | 8 | 66,981 | 68,325 | -2.01% [-14.69%～19.74%] |
| 100,000 | 1 | 204,896 | 164,834 | 19.53% [16.05%～33.22%] |
| 100,000 | 8 | 69,997 | 61,592 | 11.33% [3.01%～14.37%] |

这里单线程 TTL 写入受到的影响更明显，下降约 *19%*；100,000 规模、8 写线程下也观察到约 **11.3%** 的下降。

# 问题

实验中最大的问题就是metadata持久化的问题，早期设计的时候，想着不用持久化metadata，减小设计量，但是后来测试发现如果启动数据库的时候遍历操作来重建是不合理的，因为memtable重建通过遍历WAL，工作量有限，但是SST只是放回，没有遍历的操作，如果我们额外遍历，开销是不小的
所以后来为了解决这个问题，为memtable和FileMetaData都维护了自己的metadata，但都只维护了：

```
struct SourceMetadata {
  uint64_t total_entries = 0;
  uint64_t distinct_keys = 0;
};
```

后来发现，如果只维护两个int，sst是没法重建的，因为键会在不同的SST之间交叉重复
优化成：

```
全局对象保存：
total_entries_
key_references_  // 用户键 → 内部记录数
局部对象
1.memtable metadata
struct SourceMetadata {
  uint64_t total_entries = 0;
  uint64_t distinct_keys = 0;
};

2.SST FileMetaData
  SourceMetadata metadata;
  bool metadata_known = false;  // Legacy MANIFEST records have no summary.
  std::shared_ptr<const FileKeyReferences> key_references;
  bool has_ttl = false;
```

# AI声明

- AI完全帮助完成补充测试，性能分析
- AI帮助完成实现思路，修改位置和功能设计，我根据AI提供的TODO完成功能的代码实现，对代码掌控感还是比较强的